Question 1

GRAPH 1 → Equivalent Regular Expression

Step 1: Read the graph carefully

States

- One **start state** (marked −)
- Two **final states** (marked +)
- One intermediate non-final state at the bottom

Transitions

From the **start state**:

- Self-loop on `a, b`
- Transition to **right final state** (unlabeled → ε)
- Transition to **left final state** (unlabeled → ε)
- Transition to bottom state on `ab`

From **bottom state**:

- Transition back to start state on `ba`

Step 2: Understand the structure

Loop behavior

- At the start state, `(a + b)*` can be generated due to the self-loop.

Cycle through bottom state

- You can go:

  start ─ab→ bottom ─ba→ start

This cycle produces the string `abba`

Repeating it gives `(abba)*`

Acceptance

- From the start state, you can go to a **final state using ε**

- So **any string generated at the start state is accepted**

Step 3: Combine all behaviors

At the start state, the language consists of:

- Any number of `a` and `b`
- With optional repetitions of `abba`

So the complete language is:

$$(a + b + abba)^*$$

This can be **simplified**, because `abba` itself is already composed of `a` and `b`.

Final Regular Expression (Graph 1):

$$\boxed{(a + b)^*}$$

GRAPH 2 → Equivalent Regular Expression

Step 1: Read the graph

States

- Two states
- **Both are final states**

Transitions

- Left state:
    - Self-loop on `a, b`
    - Transition to right state on `baa`
- Right state:
    - Self-loop on `a, b`
    - Transition back to left state on `abb`

Step 2: Interpret self-loops

- At **both states**, self-loop on `a, b` means:

$$(a + b)^*$$

- This can happen **before, after, or between transitions**.

Step 3: Interpret cross transitions

- Left → Right gives `baa`
- Right → Left gives `abb`

Together, one full round trip gives:

$$baa\ abb$$

$$(baa\ abb)^*$$

Step 4: Construct full expression

Since:

- We can start at either final state
- We can loop freely on `(a+b)*`
- We can alternate states using `baa` and `abb`

The complete expression is:

$$(a + b)^*(baa\ abb)^*(a + b)^*$$

Final Regular Expression (Graph 2)

$$\boxed{(a+b)^*(baa\,abb)^*(a+b)^*}$$

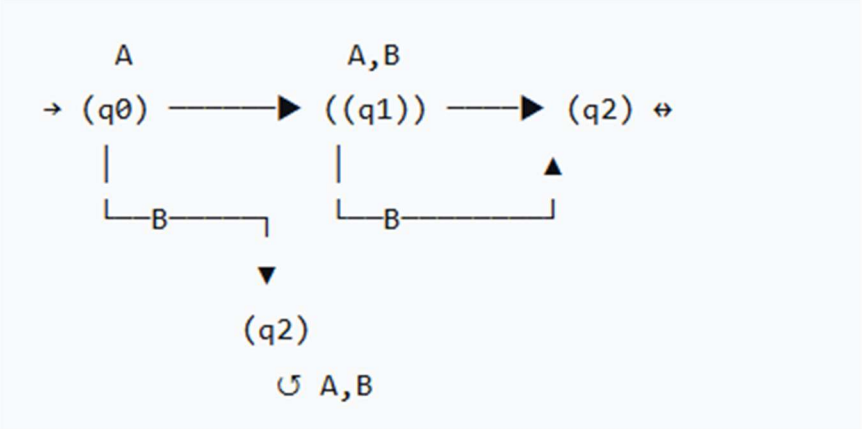
Question 2

1-FA that accepts only the word "A"

Interpretation: Alphabet Σ = {A, B}, only string "A" accepted.

Step-by-step:

1. Need states to remember:
   - Haven't started (q0)
   - Read exactly "A" (q1-final)
   - Read wrong first letter or too long (q2-trap)
2. **Complete transition table:**
   - δ (q0, A) = q1
   - δ(q0, B) = q2
   - δ(q1, A) = q2
   - δ(q1, B) = q2
   - δ(q2, A) = q2
   - δ(q2, B) = q2
3. **ASCII Diagram:**



Accepting path: q0→A→q1
Rejecting: Any other input length or wrong letter.

2 FA with three states that accepts all words

Interpretation: Σ = {a, b}, L = Σ*

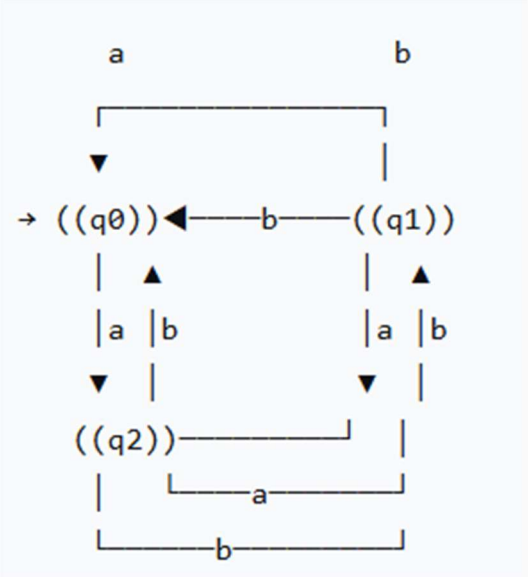
Step-by-step:

1. To accept all words, all **reachable states must be final**.
2. Use 3 states all final.
3. Ensure total transition function.

Complete transitions:

- From q0: a→q1, b→q2
- From q1: a→q2, b→q0
- From q2: a→q0, b→q1

ASCII Diagram:



Verification: Every input string ends in some state, all are final.

3-FA that accepts only "baa", "ab", and abb and NO other strings longer or shorter?

Step 1 — List accepted strings and their paths:

1. `ab`: Path: a → b
2. `baa`: Path: b → a → a
3. `abb`: Path: a → b → b

Step 2 — Design states logically:

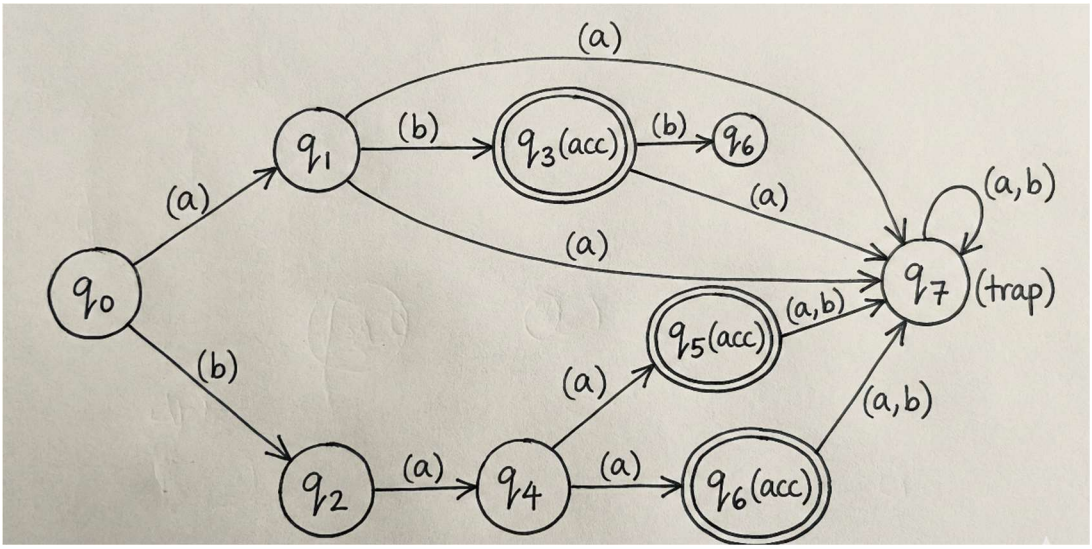
We need to distinguish prefixes and ensure only exact strings accepted.

Let's define states:

- **q0**: start
- **q1**: after `a`
- **q2**: after `b`
- **q3**: after `ab` → **accepting** for `ab`
- **q4**: after `ba`
- **q5**: after `baa` → **accepting** for `baa`
- **q6**: after `abb` → **accepting** for `abb`
- **q7**: trap (dead state)

State	a	b	Notes
q0	q1	q2	start
q1	q7	q3	after `a`
q2	q4	q7	after `b`
q3	q7	q6	after `ab` (accept)
q4	q5	q7	after `ba`
q5	q7	q7	after `baa` (accept)
q6	q7	q7	after `abb` (accept)
q7	q7	q7	trap

Accepting states: F = {q3, q5, q6}



Step 5 — Verification:

Accepted strings:

1. `ab`:
   q0 →a→ q1 →b→ q3 accept
2. `baa`:
   q0 →b→ q2 →a→ q4 →a→ q5 accept
3. `abb`:
   q0 →a→ q1 →b→ q3 →b→ q6 accept

Rejected examples:

- `a`: q0→q1 not final
- `b`: q0→q2
- `aba`: q0→q1→q3→a→q7
- `abba`: q0→q1→q3→q6→a→q7

- `ba`: q0→q2→q4
- `bab`: q0→q2→q4→b→q7
- `ab`: but `ab`+anything → q7

Step 5 — Verification:

Accepted strings:

1. `ab`:
   q0 →a→ q1 →b→ q3 accept
2. `baa`:
   q0 →b→ q2 →a→ q4 →a→ q5 accept
3. `abb`:
   q0 →a→ q1 →b→ q3 →b→ q6 accept

Rejected examples:

- `a`: q0→q1 not final
- `b`: q0→q2
- `aba`: q0→q1→q3→a→q7
- `abba`: q0→q1→q3→q6→a→q7
- `ba`: q0→q2→q4
- `bab`: q0→q2→q4→b→q7
- `ab`: ok but `ab`+anything → q7

Question 3

PART (i)

Language L1

All words that **begin and end with the same doubled letter**, either of the form
aa … aa or **bb … bb**

Note: `aaa` and `bbb` are **NOT** in the language.

Step 1: Understand the constraints

What does “doubled letter” mean?

- `aa` or `bb` (two identical letters together)

So valid words must:

1. **Start with `aa` and end with `aa`**, OR
2. **Start with `bb` and end with `bb`**
3. Length **must be ≥ 4**
4. Middle part can be **anything** (including empty)
5. Words like:
    - `aaa` (starts with `aa` but ends with `aa`? No — overlap)
    - `bbb`

Accepted:

- `aaaa`
- `aabaa`
- `aabbbaa`
- `bbbb`
- `bbabbb`

Rejected:

- `aaa`
- `bbb`
- `aa`
- `bb`
- `aab`
- `bbaa`

Step 2: Strategy to build the TG

We must:

- First **remember** whether we started with `aa` or `bb`
- Then allow **any symbols** in the middle
- Finally accept **only if we end with the same doubled letter**

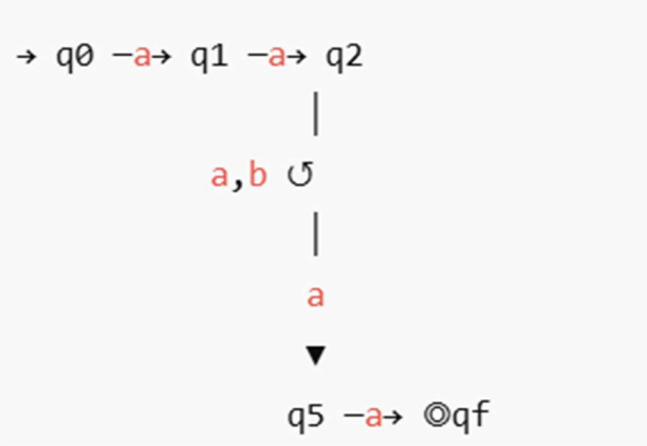
This requires **memory**, so we use states.

Transition Graph:

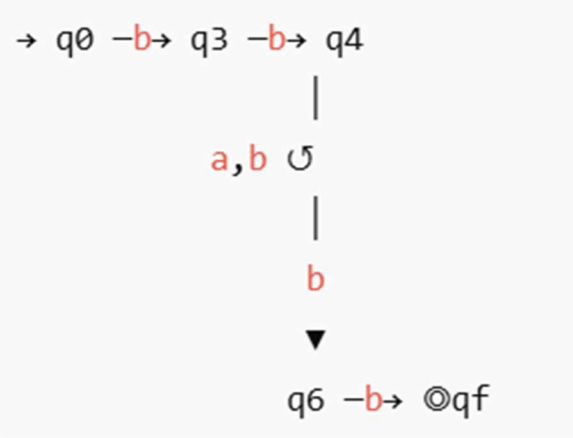
State	Meaning

q0	Start
q1	Seen first `a`
q2	Confirmed start with `aa`
q3	Seen first `b`
q4	Confirmed start with `bb`
q5	One `a` before final `aa`
q6	One `b` before final `bb`
qf	Final (accepting)

TG for **aa…aa:**



TG for **bb…bb:**



PART (ii)

Prepare a TG that accepts **all strings that end in a word from L1**

Step 1: Understand what this means

Language:

$$L_2 = \Sigma^* L_1$$

That is:

- Any string over `{a, b}`
- As long as the **suffix** belongs to L1L_1L1

Examples

Accepted:

- `aabbaa`
- `bbbbaaaa`
- `ababbbabb`

Rejected:

- `aba`
- `aa`
- `bbb`

Step 2: Key idea

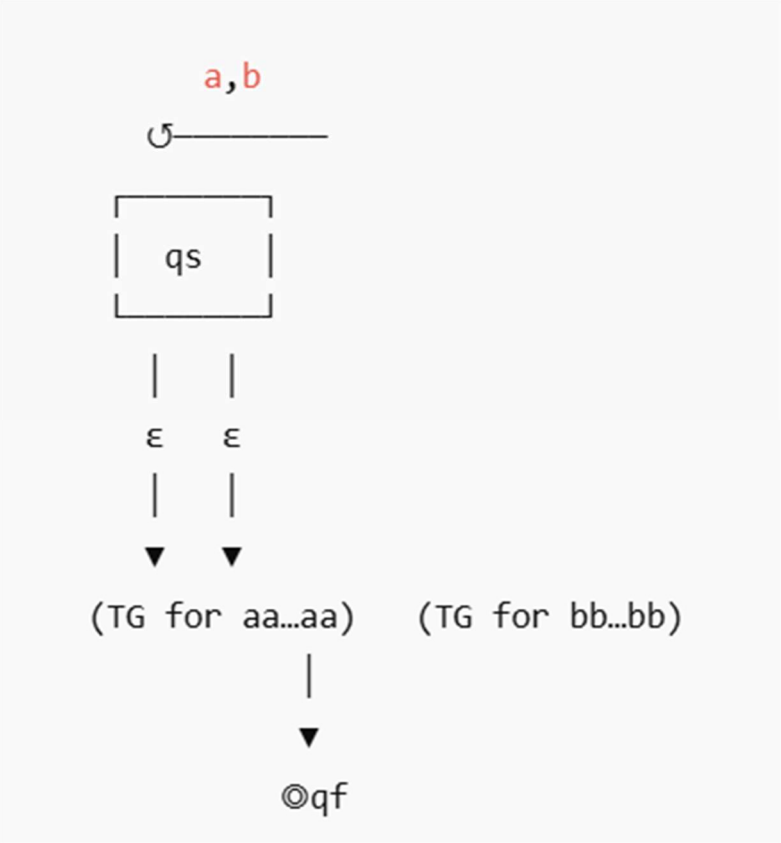
To accept **strings ending in L_1**:

- We allow **any prefix**
- Then non-deterministically guess the start of `aa…aa` or `bb…bb`

This is done by:

- Adding a **loop on start state**
- Using **ε-transitions** into the TG from part (i)

Transition Graph



- qs allows **any prefix**
- ε-transitions guess the start of a valid suffix
- Acceptance only if the suffix satisfies part (i)

Question 4

Given Transition Graph (TG)

States

- State **2** → start state

- State **4** → final state
- State **3** → intermediate state

From	To	Label
2	2	r_1
2	3	r_3
3	2	r_4
3	3	r_5
2	4	r_2
4	2	r_6
3	4	r_7
4	3	r_8
4	4	r_9

Goal

Convert the TG into a single state and find the equivalent Regular Expression using state elimination.

Step 1: State Elimination Rule (VERY IMPORTANT)

When eliminating an intermediate state k, for every pair of states i → k → j, update the transition as:

$$R_{ij}^{new} = R_{ij} + R_{ik}(R_{kk})^* R_{kj}$$

Step 2: Choose the State to Eliminate

We eliminate state 3 (intermediate state).

Remaining states will be:

- State 2 (start)
- State 4 (final)

Step 3: Update Transitions After Eliminating State 3

We now compute new transitions between 2 and 4, including loops.

New loop on State 2

Original loop:

- r_1

New paths through state 3:

- $2 \to 3 \to 2$
- Label: $r_3\ (r_5)^*\ r_4$

Updated loop on state 2:

$$r_1 + r_3(r_5)^* r_4$$

New loop on State 4

Original loop:

- r_9

New paths through state 3:

- $4 \to 3 \to 4$
- Label: $r_8\ (r_5)^*\ r_7$

Updated loop on state 4:

$$r_9 + r_8(r_5)^* r_7$$

New transition from State 2 to State 4

Original transition:

- r_2

New path via state 3:

- $2 \rightarrow 3 \rightarrow 4$
- Label: `r3 (r5)^* r7`

$$r_2 + r_3(r_5)^* r_7$$

New transition from State 4 to State 2

Original transition:

- `r6`

New path via state 3:

- $4 \rightarrow 3 \rightarrow 2$
- Label: `r8 (r5)^* r4`

Updated transition:

$$r_6 + r_8(r_5)^* r_4$$

Step 4: Reduced TG (Only States 2 and 4)

Now we have:

- Loop on **2**:

$$A = r_1 + r_3(r_5)^* r_4$$

- Loop on **4**:

$$D = r_9 + r_8(r_5)^* r_7$$

- Transition **2 → 4**:

$$B = r_2 + r_3(r_5)^* r_7$$

- Transition **4 → 2**:

$$C = r_6 + r_8(r_5)^* r_4$$

Step 5: Final Regular Expression

$$A^* B (D^* C A^* B)^* D^*$$

Step 6: Substitute Values:

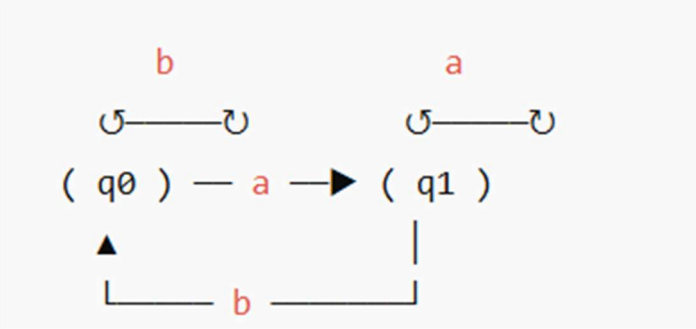
$$(r_1 + r_3 r_5^* r_4)^* (r_2 + r_3 r_5^* r_7) \left((r_9 + r_8 r_5^* r_7)^* (r_6 + r_8 r_5^* r_4) (r_1 + r_3 r_5^* r_4)^* (r_2 + r_3 r_5^* r_7)\right)^* (r_9 + r_8 r_5^* r_7)^*$$

FINAL ANSWER:

$$(r_1 + r_3 r_5^* r_4)^* (r_2 + r_3 r_5^* r_7) \left((r_9 + r_8 r_5^* r_7)^* (r_6 + r_8 r_5^* r_4) (r_1 + r_3 r_5^* r_4)^* (r_2 + r_3 r_5^* r_7)\right)^* (r_9 + r_8 r_5^* r_7)^*$$

Question 5

FA_1 (top automaton)



Details

- **q0** → start state (non-final)
- **q1** → final state (+)
- Transitions:
    - q0 ─a→ q1
    - q0 ─b→ q0
    - q1 ─a→ q1
    - q1 ─b→ q0

Step-by-step language description

- From **q0**, you can read any number of `b`'s (stay in q0)
- To reach final state **q1**, you must read **one** `a`
- Once in **q1**:

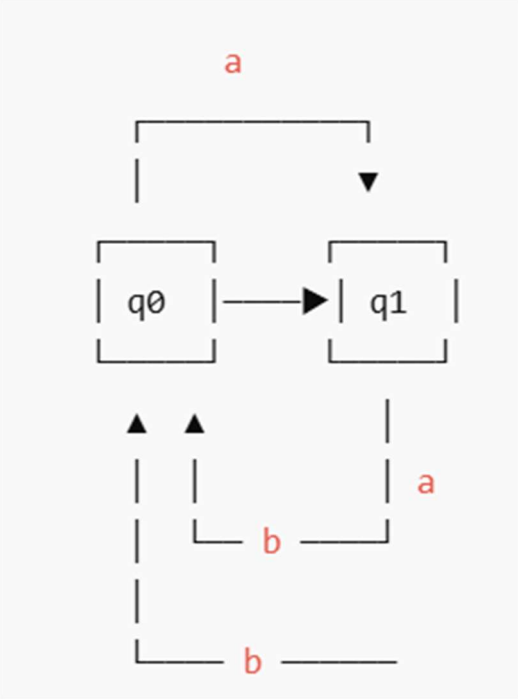
- `a` keeps you in q1
- `b` sends you back to q0 (must read `a` again to accept)

A string is **accepted iff it ends with `a`**

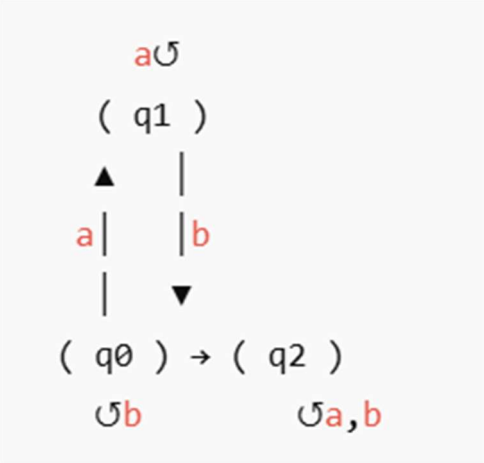
Regular Expression for FA_1

All strings over `{a, b}` that **end in `a`**:

$$\boxed{r_1 = (a+b)^*a}$$



PART 2: FA_2 (bottom automaton)



Details

- **q0** → start state (non-final)
- **q1** → intermediate
- **q2** → final state (+)

Transitions:

- q0 ─b→ q0
- q0 ─a→ q1
- q1 ─a→ q1
- q1 ─b→ q2
- q2 ─a→ q2
- q2 ─b→ q2

tep-by-step language description

1. Start at q0:
    - Any number of `b`'s allowed
2. Read **at least one `a`** → reach q1
3. Stay in q1 with more `a`'s
4. Read **one `b`** → reach final state q2
5. After reaching q2, **anything is allowed**

Strings that contain:

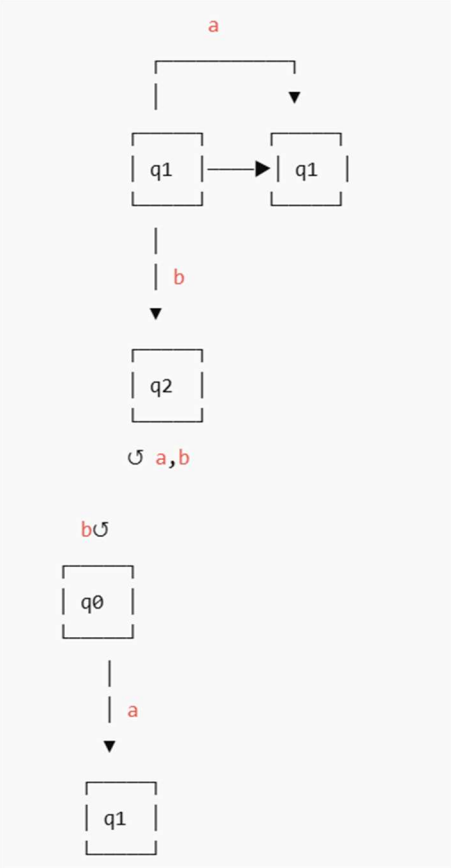
(at least one a) followed by (at least one b)

Regular Expression for FA_2

Break it down:

- `b*` → before first `a`
- `a+` → one or more `a`
- `b` → transition to final
- `(a+b)*` → anything after acceptance

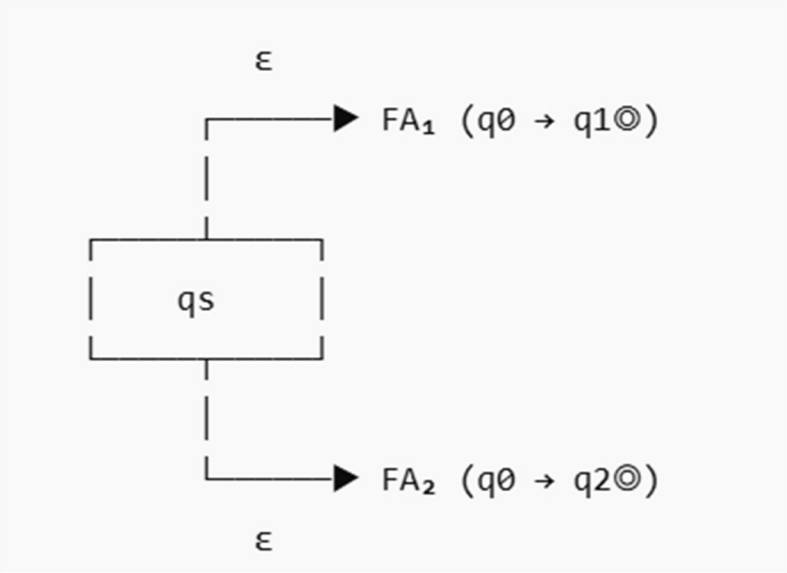
$$\boxed{r_2 = b^* a^+ b(a+b)^*}$$



FA for $r_1 + r_2$ (Union)

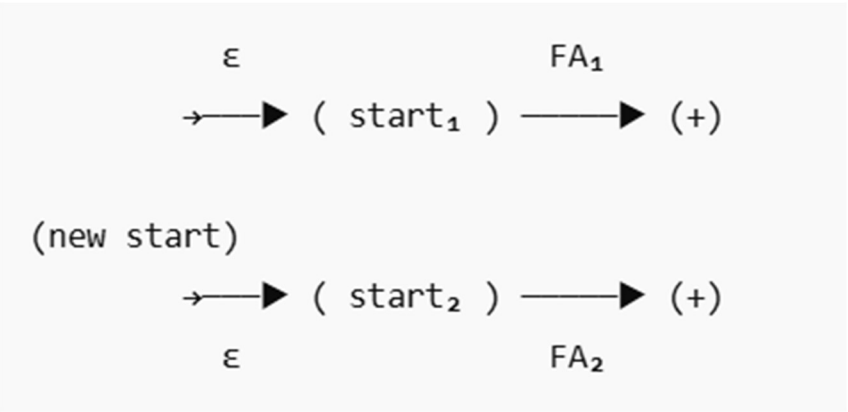
Regular Expression :

$$r_1 + r_2 = (a+b)^* a \ + \ b^* a^+ b(a+b)^*$$



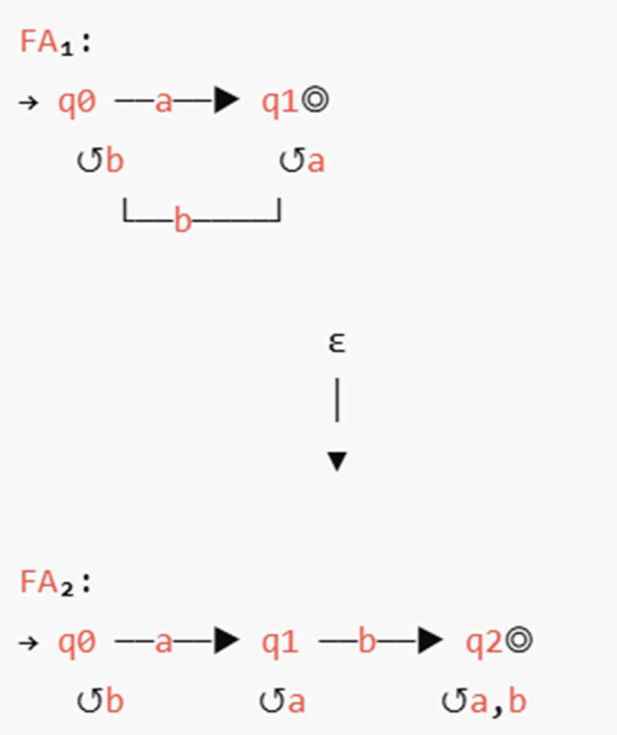
FA construction idea

- Create a **new start state**
- ε-transition to:
    - FA_1
    - FA_2
- Accept if **either accepts**



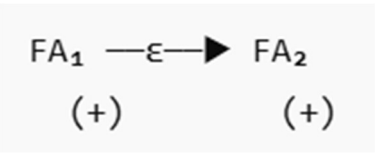
FA for r_1 r_2 (Concatenation)

$$r_1 r_2 = (a+b)^* a\, b^* a^+ b (a+b)^*$$



FA construction idea

1. Take FA_1
2. Add ε-transition from **final state of FA_1** to **start of FA_2**
3. Final states = final states of FA_2



This ensures:

- First part matches r_1
- Second part matches r_2

Regular Expressions

$$\boxed{r_1 = (a+b)^*a}$$

$$\boxed{r_2 = b^*a^+b(a+b)^*}$$